

Assignment 2

Working with Data in Python: NumPy and Pandas

Maximum score: 100 points | 4 tasks × 25 points

Upload to Moodle by 03.10.2026 and complete the oral defense on 09.02.2026

Report uploaded after 03.10.2026: -20%

Defense completed after 09.02.2026: -20%

General Requirements

- Complete the work in a Jupyter Notebook (.ipynb). All cells must run sequentially from top to bottom without errors (Run All).
- For reproducibility of random data, set a seed at the beginning of Tasks 1 and 2, for example `np.random.seed(42)`, or use `np.random.default_rng(42)`.
- In Tasks 1–2, perform the main calculations with NumPy. Do not replace vectorized operations with ordinary loops when the task can be solved using NumPy.
- Tasks 3–4 use Pandas. matplotlib and seaborn are allowed for visualization.
- After each logical block, display the result and add a brief comment explaining what was calculated and what the result means.
- Variable names must be clear and meaningful. Do not use names such as `a`, `b`, `x1`, `x2` for the main data.
- Before submission, remove unnecessary test cells, duplicated code, and unnecessary outputs.
- During the defense, the instructor may ask you to explain any line of code, modify a filtering condition, select a different slice, or recalculate a metric for another group.

Task 1. NumPy: Arrays, Statistics, and Vectorization (25 points)

Context: simulate numerical characteristics of 200 apartment sale listings and perform basic analysis using only NumPy.

1.1. Data Preparation and Creation — 4 points

Import NumPy and set a fixed seed for the random number generator.

Create `area_array` — 200 apartment area values ranging from 25 to 150 m². The values may be integers or floating-point numbers.

Create `rooms_array` — 200 integer values for the number of rooms, from 1 to 5 inclusive.

Create `price_array` — 200 apartment price values ranging from 15 to 120 million tenge.

Make sure that all three arrays contain the same number of elements and correspond to the same 200 apartments.

For each array, display shape, ndim, size, and dtype.

Hint: You may use `np.random.randint()`, `np.random.uniform()`, or methods of the `rng` object for generation. Values in `price_array` may be rounded to 1–2 decimal places.

Expected Result

- Three arrays of length 200.
- Output of the main attributes of each array.
- A brief explanation of what shape, ndim, size, and dtype mean.

1.2. Indexing, Slicing, and Boolean Filtering — 5 points

1. Display the first 10 and the last 10 elements of each array.
2. Select and display the apartment areas with indices from 20 to 39 inclusive.
3. Find the number of apartments with an area greater than 70 m².
4. Find apartments whose price is above the mean value of `price_array`. Display their count and the first 10 matching prices.
5. Create one compound Boolean mask for apartments where `area > 70` and `rooms >= 3`. Display the number of such apartments.
6. Make the condition more complex: find apartments with 2–4 rooms, an area of 60–100 m², and a price not exceeding 80 million tenge. Display the indices of the first 10 matching apartments.

Hint: For compound conditions, use `&`, `|`, and parentheses. Do not use Python `and/or` with NumPy arrays.

Expected Result

- Examples of standard indexing and slicing.
- At least three Boolean masks.
- The number of objects satisfying each condition.

1.3. Descriptive Statistics — 6 points

1. For `area_array` and `price_array`, calculate mean, median, min, max, standard deviation, and variance.
2. For both arrays, calculate the 10th, 25th, 50th, 75th, 90th, and 99th percentiles.
3. Determine the index of the most expensive apartment. Using this index, display its price, area, and rooms.
4. Determine the index of the apartment with the largest area and display all of its characteristics.
5. Count how many apartments have a price below the median and how many have an area above the mean.
6. Compare the mean and median prices in 1–2 sentences. Explain what a noticeable difference between them may indicate.

Hint: You will need `np.mean`, `np.median`, `np.min`, `np.max`, `np.std`, `np.var`, `np.percentile`, and `np.argmax`.

1.4. Vectorized Calculations and New Features — 6 points

1. Create `price_per_m2_array` using the formula `price_array / area_array`. You may not use a for loop for this calculation.
2. Find the mean, median, minimum, and maximum price per 1 m².
3. Determine the index of the apartment with the maximum price per 1 m² and display its original characteristics.
4. Count the number of apartments whose price per 1 m² is above the mean value.
5. Create a conditional category `expensive_flag` as a Boolean array: True if the apartment price is above the 75th percentile of `price_array`; otherwise, False.
6. Check how many True and False elements `expensive_flag` contains.

Expected Result

- A `price_per_m2_array` of length 200.
- A Boolean array `expensive_flag`.
- Statistics for the new feature calculated without loops.

1.5. Normalization and Brief Conclusion — 4 points

1. Normalize `area_array` and `price_array` using Min-Max normalization to the range [0, 1]. Implement the formula yourself using NumPy.

2. Check the minimum and maximum values of the normalized arrays.
3. Display the first 5 original values and the first 5 normalized values.
4. Write 2–3 sentences explaining why normalization is needed and in which tasks it can be useful.

Task 2. NumPy: Multidimensional Arrays and Tensor Analysis (25 points)

Context: hourly temperatures are available for 3 cities over 7 days. The data are represented as a three-dimensional NumPy array.

2.1. Creating and Examining a Tensor — 4 points

1. Create a tensor `T` with shape (7, 24, 3): axis 0 — days, axis 1 — hours, axis 2 — cities.
2. Fill the tensor with random temperatures from -10 to +35 °C.
3. Display shape, ndim, size, and dtype.
4. Separately explain what each axis represents and state the values of `T.shape[0]`, `T.shape[1]`, and `T.shape[2]`.

Expected Result

- A tensor with shape (7, 24, 3).
- A correct explanation of the axes.

2.2. Indexing and Slicing — 6 points

1. Display the temperatures of the second city for all days and all hours. State the shape of the resulting array.
2. Display all data for the first day for all three cities. State the shape.
3. Retrieve one value: the temperature on the 3rd day, at 12:00, in the 2nd city. Remember that indexing starts from zero.
4. Display the temperatures of all cities from 09:00 to 18:00 for the entire week.
5. Display the temperatures of the first city for the weekend days (6th and 7th days).
6. Obtain an array of temperatures for all cities exactly at 08:00 over 7 days and state its shape.

Hint: During the defense, be ready to explain why `T[:, :, 1]`, `T[0, :, :]`, and `T[:, 8, :]` have different shapes.

2.3. Aggregation Along Axes — 7 points

1. Find the mean temperature for each city over the entire week.
2. Find the minimum and maximum temperatures for each city.
3. For each day, find the maximum temperature across all hours and cities.
4. For each day, calculate the mean temperature separately for each city. The result must have shape (7, 3).
5. Determine the hour of the day with the lowest mean temperature by averaging across all days and cities.
6. Determine the day with the highest mean temperature.
7. Determine the city with the highest mean temperature for the week.

Hint: In this section, it is important to select axis correctly. In your comments, indicate which axes are reduced in each calculation.

2.4. Reshaping and Data Verification — 4 points

1. Reshape `T` from (7, 24, 3) into a matrix with shape (7, 72).
2. Explain what is now contained in one row of the new matrix.

3. Reshape the matrix back to (7, 24, 3).
4. Check whether the original and restored tensors are identical using `np.array_equal()` or `np.allclose()`.

2.5. Comparing Cities and Interpretation — 4 points

1. Transform the data for each city into a sequence of 168 temperatures (7×24).
2. Build a 3×3 correlation matrix using `np.corrcoef`.
3. Ignoring the diagonal, identify the pair of cities with the highest correlation coefficient.
4. Write 2–3 sentences explaining what positive, negative, and near-zero correlation mean in the context of temperatures.

Task 3. Pandas: Data Loading, Cleaning, and Preparation (25 points)

Context: work with a dataset of apartment sale listings. Use the CSV files or a single CSV file provided by the instructor.

3.1. Loading and Initial Exploration — 4 points

1. Import pandas, numpy, and the necessary visualization libraries.
2. If the instructor provided several CSV files, read them and combine them into one DataFrame using `pd.concat(..., ignore_index=True)`. If one file was provided, simply load it.
3. Display the first 5 and last 5 rows of the DataFrame.
4. Display `df.shape`, `df.columns`, `df.dtypes`, `df.info()`, and `df.describe(include="all")`.
5. In 2–3 sentences, describe the size of the dataset and the types of features it contains.

3.2. Checking and Handling Duplicates — 4 points

1. Count the number of fully duplicated rows using `duplicated().sum()`.
2. Display several detected duplicates, if any exist.
3. Check for duplicates based on logically important columns. For example: ID + Price, Address + Price, or another combination appropriate for your dataset.
4. Decide whether duplicates should be removed. You must explain your decision.
5. If duplicates are removed, show the number of rows before and after removal.

Hint: Do not remove rows automatically just because `duplicated()` returned `True`. First explain why the rows should actually be considered repeated listings.

3.3. Missing Values — 6 points

1. Calculate missing values for each column: both the count and percentage.
2. Sort the columns by the number of missing values in descending order.
3. Select at least three columns with missing values and propose a handling method for each one.
4. For a numerical column, apply median or group-based imputation if justified.
5. For a categorical column, apply mode imputation or fill with a separate category if justified.
6. If any column has too many missing values, you may justify dropping it. The reason must be stated.
7. After handling missing values, display the missing-values table again.

Expected Result

- A table/Series showing the number and percentage of missing values.
- A justification for the selected imputation method for each processed column.
- A verification after cleaning.

3.4. Data Types and Conversions — 4 points

1. Identify columns that should be numeric but have dtype object.
2. Remove text characters, spaces, or units of measurement that prevent conversion, if present.

3. Convert such columns using `pd.to_numeric(..., errors="coerce")`.
4. If there is a date column, convert it to datetime using `pd.to_datetime()`.
5. After the conversions, display dtypes again and check whether new NaN values have appeared.

3.5. Renaming Columns and Feature Engineering — 5 points

1. Rename long column names to short, consistent, and clear names.
2. Create `price_per_m2 = price / area`. First make sure that area does not contain zero values.
3. Create `floor_category`, for example: first / middle / last. If the data structure is different, an equivalent categorical feature is allowed.
4. Create at least one more meaningful feature: `area_category`, `price_category`, `room_area_ratio`, `building_age`, or another feature that can be derived from the existing data.
5. For each new feature, write a brief explanation of what it represents.

3.6. Saving and Quality Control — 2 points

1. Save the cleaned DataFrame as `apartments_clean.csv` without the index.
2. Read the saved file again into a new DataFrame and compare its shape with the original cleaned DataFrame.
3. Display the first 3 rows of the saved file as confirmation.

Task 4. Pandas: Analysis, Grouping, and Visualization (25 points)

Use only the cleaned DataFrame from Task 3. The goal is to obtain meaningful analytical insights, not merely to create plots.

4.1. Descriptive Statistics and Filtering — 5 points

1. Display `describe()` for the numerical columns.
2. Find the 5 most expensive apartments and display at least four informative columns for them.
3. Find the 5 apartments with the lowest price per 1 m².
4. Create a filter with two conditions, for example `rooms >= 3` and `area > 80`. Display the number of rows and the first matching records.
5. Create another filter with three conditions of your choice and explain what type of apartment it selects.

4.2. GroupBy and Aggregation — 7 points

1. Calculate the mean and median price by number of rooms.
2. Calculate the mean price per 1 m² by district or residential complex. Sort the result in descending order.
3. For a selected categorical column, use `groupby().agg()` to obtain count, mean, median, min, and max for price.
4. Find the group with the highest mean price and the group with the lowest mean price.
5. Create a `pivot_table` using two categorical features if the relevant columns are available. For example: rows — district, columns — number of rooms, values — mean price.
6. Write at least two conclusions based on the groupings.

Hint: If your dataset does not contain a district or residential complex column, choose another meaningful categorical column and state this in the Notebook.

4.3. Visualization — 8 points

1. Create a bar chart of mean price by number of rooms.
2. Create a scatter plot showing the relationship between price and area.
3. Create a boxplot of the price distribution by number of rooms or another appropriate category.

4. Create a histogram for price_per_m2 or price.
5. Create a heatmap of the correlation matrix of numerical features.
6. Each plot must have a title, axis labels, and a readable size. For plots with categories, rotate the X-axis labels if necessary.

Expected Result

- At least five plots.
- Each plot has a title and axis labels.
- After each plot, provide 1–2 sentences of interpretation.

4.4. Correlation Analysis and Final Conclusions — 5 points

1. From the correlation matrix, select the features most strongly associated with price. State the coefficients.
2. Separately identify one feature with a positive correlation and one with a negative correlation, if such features are present.
3. Do not infer causality based on correlation alone.
4. Formulate at least 5 meaningful conclusions from the entire Pandas work. At least two conclusions must be based on groupings, and at least two must be based on visualizations or correlation.

Submission Requirements

- One Jupyter Notebook (.ipynb) containing all four completed tasks.
- The apartments_clean.csv file obtained after Task 3.
- All tables and plots must be displayed inside the Notebook after Run All.
- At the end of the Notebook, include a Conclusions section with brief conclusions for Tasks 1–4.
- Before uploading, make sure the Notebook opens and runs without referring to absolute paths on your computer.

Assessment Criteria

Component	What is assessed	Points
Task 1	Correct NumPy arrays, indexing, Boolean masks, statistics, vectorization, and normalization	25
Task 2	Working with a 3D array, slicing, axis, aggregation, reshape, and correlation	25
Task 3	Loading, duplicates, missing values, data types, feature engineering, and saving	25
Task 4	Filtering, GroupBy/Pivot, visualization, correlation analysis, and conclusions	25
Total		100

Defense Questions

Important: The questions will not necessarily be asked in the same order. The instructor may ask you to explain code, modify a specific condition, or perform a small analysis task during the defense.

A. NumPy

1. How is a NumPy array different from a regular Python list?
2. What do the attributes shape, ndim, size, and dtype represent?

3. Why are NumPy arrays usually more efficient than Python lists for numerical computations?
4. What is vectorization? Give an example from your Task 1.
5. What is broadcasting in NumPy, and when is it used?
6. How does standard indexing differ from slicing?
7. What does `arr[10:20]` return, and is index 20 included?
8. How can you obtain the last 10 elements of a one-dimensional array?
9. What is a Boolean mask, and what type of elements does it contain?
10. Why are `&` and `|` used to combine conditions for NumPy arrays instead of `and` and `or`?
11. How can you filter apartments simultaneously by area, number of rooms, and price?
12. How does `np.mean()` differ from `np.median()`? In what case can the median be more informative?
13. What does standard deviation measure?
14. How is variance related to standard deviation?
15. What is a percentile? What does the 75th percentile of price mean?
16. What is `np.argmax()` used for? What exactly does it return?
17. Why can `price_per_m2_array` be calculated without a loop?
18. What does Min-Max normalization mean, and what range will the values fall into?
19. What happens during Min-Max normalization if `max == min`?
20. What is a multidimensional array or tensor?
21. What do the three axes of tensor `T` with shape `(7, 24, 3)` represent?
22. What result and shape will `T[:, :, 1]` produce?
23. What result and shape will `T[0, :, :]` produce?
24. How can you obtain the temperatures of all cities at 8:00 for all days?
25. What does the `axis` parameter mean in `np.mean()`, `np.max()`, and `np.min()`?
26. Which axes should be averaged to obtain one mean temperature for each city?
27. How can you obtain a matrix with shape `(7, 3)` containing the mean temperature of each city for each day?
28. How does `reshape()` differ from `transpose()`?
29. Why does the number of elements remain unchanged after `reshape(7, 72)`?
30. What can `np.array_equal()` or `np.allclose()` be used for?
31. What does the correlation coefficient measure?
32. What is the range of the correlation coefficient?
33. What does a correlation close to `+1`, `-1`, and `0` mean?
34. Why does a high correlation not prove a cause-and-effect relationship?

B. Pandas

1. What are `Series` and `DataFrame`? How do they differ?
2. What are `head()`, `tail()`, `shape`, `columns`, and `dtypes` used for?
3. What information does `df.info()` display?
4. How does `df.describe()` differ from `df.info()`?
5. What is `pd.concat()` used for, and why might `ignore_index=True` be needed?
6. What is a fully duplicated row?
7. How does `duplicated()` differ from `drop_duplicates()`?
8. Why should duplicates not always be removed automatically?
9. How can you check for duplicates using only selected columns?
10. How can you determine the number of missing values in each column?

11. How can you calculate the percentage of missing values by column?
12. What is the difference between `dropna()` and `fillna()`?
13. When is it reasonable to fill numerical missing values with the mean, and when with the median?
14. When is mode appropriate for a categorical feature?
15. What is group-based imputation, and why can it sometimes be better than using the overall mean?
16. What does `pd.to_numeric(errors="coerce")` do?
17. Why is `pd.to_datetime()` used?
18. Why should NaN values be checked again after data type conversion?
19. How can you rename one or more columns in Pandas?
20. What is feature engineering? Name the features you created in Task 3.
21. Why should zero area values be checked before calculating `price_per_m2`?
22. What is `loc[]` used for, and how does it differ from `iloc[]`?
23. How can you filter DataFrame rows using several conditions at once?
24. What does `sort_values()` do?
25. What does `groupby()` do, and what are the main stages of the split-apply-combine operation?
26. Why is `agg()` more useful than a single `mean()` after `groupby()`?
27. What is a `pivot_table`, and how does it differ from a regular `groupby()`?
28. What does count represent in an aggregated table, and why is it important when comparing groups?
29. Why should the mean of a group with a very small count be interpreted cautiously?
30. What are bar charts, scatter plots, boxplots, and histograms used for? What information does each one show?
31. What can you observe in a scatter plot of price versus area?
32. What does a boxplot show, and what are outliers?
33. What does a histogram show?
34. What is a correlation matrix?
35. Why are the diagonal values in a correlation heatmap equal to 1?
36. Can Pearson correlation be applied to arbitrary text columns? Why or why not?
37. Why does correlation by itself not imply causation?
38. How can you save a DataFrame to CSV without an additional index column?
39. How can you check that a saved CSV file was read correctly?
40. How do `merge()`, `join()`, and `concat()` differ in purpose?
41. What happens if `groupby` is applied to a column containing NaN values?
42. How can you find the 5 rows with the highest or lowest values of a particular feature?
43. What major data quality problems did you identify in your dataset?
44. Which data transformation in your work had the greatest impact on the final analysis, and why?